

# 第一部分：《白话机器学习算法》第1章 读书笔记

## 一、数据科学研究的核心步骤

### 1. 准备数据

数据格式：通常为表格形式，行代表数据点（观测结果），列代表变量（属性/特征/维度）。

变量类型：

二值变量：只有两种可能的值（如：是否买鱼）。

分类变量：有两个以上的类别（如：顾客类别）。

整型变量：用整数表示（如：水果购买量）。

连续变量：最精细，用小数表示（如：支出金额）。

变量选择：从众多变量中筛选出与研究目标最相关的变量，避免计算缓慢或预测干扰。

特征工程：通过重新编码（如将动物分为食草/食肉动物）或降维（如主成分分析）创造更有效的变量。

缺失数据处理：

近似：分类变量用众数替换；连续变量用中位数替换。

计算：使用监督学习算法估算缺失值（更准确但耗时）。

移除：万不得已时删除含缺失值的行（可能减少数据量或引入偏差）。

- 2. 选择算法

- 无监督学习：发现数据中的隐藏模式（如：顾客分组、商品搭配）。
- 常用算法：k均值聚类、主成分分析、关联规则、社会网络分析。
- 监督学习：基于已有模式进行预测。
- 回归问题：预测连续值（如：水果购买量）。
- 分类问题：预测二值或分类值（如：是否会买鱼）。
- 常用算法：回归分析、k最近邻、支持向量机、决策树、随机森林、神经网络。
- 强化学习：根据反馈结果不断改进模型（如：优化广告投放策略）。

- 3. 参数调优

- 目的：调整算法设置，优化模型准确度。
- 常见问题：
- 过拟合：模型对数据中的随机波动过度敏感，泛化能力差。
- 欠拟合：模型过于简单，忽视了数据中的基本模式。
- 正则化：引入惩罚参数，控制模型复杂度，提高泛化能力。

- 4. 评价模型

- 分类指标：
  - 预测准确率：正确预测的比例。
  - 混淆矩阵：区分假正例和假负例，揭示模型优缺点。
- 回归指标：
  - 均方根误差：放大较大误差，对异常值敏感。
- 验证方法：
  - 训练集/测试集划分：模拟新数据，评估模型泛化能力。
  - 交叉验证：将数据集分成若干组，反复训练和测试，取平均结果。

# 第二部分：PPT“本章概念汇总”整理笔记

- 一、核心概念定义

- | ● 概念     | 定义                                                                                  |
|----------|-------------------------------------------------------------------------------------|
| ● 大数据    | 规模巨大、形式多样、蕴含丰富信息的数据集合。具备4V特征：Volume（体量大）、Variety（类型多）、Value（价值密度低）、Velocity（处理速度快）。 |
| ● 数据挖掘   | 从大量、不完全、有噪声的数据中，提取隐含的、未知但有潜在价值的信息和知识的过程。                                            |
| ● 数据降维   | 减少变量数量，提取最具代表性的信息，常用方法包括主成分分析（PCA）、t-SNE等。                                          |
| ● 监督学习   | 使用有标签的数据进行训练，进行预测或分类。                                                               |
| ● 无监督学习  | 使用无标签数据，探索数据内在结构或隐藏模式。                                                              |
| ● 结构化数据  | 以表格形式存储的数据，如数据库中的行和列。                                                               |
| ● 半结构化数据 | 具有一定结构但不严格符合表格形式，如XML、JSON。                                                         |
| ● 非结构化数据 | 无固定格式的数据，如文本、图片、音频、视频。                                                              |
| ● 数据预处理  | 包括数据清洗、集成、变换、归约等步骤，目的是提高数据质量。                                                       |
| ● 数据标准化  | 将数据统一到同一尺度（如使用百分位数），便于比较和分析。                                                        |
| ● 衍生变量   | 通过原始变量组合或转换得到的新变量（如：将“维生素C”和“膳食纤维”组合）。                                              |

- 二、算法分类概览
  - 算法类别          任务目标          常用算法
  - 无监督学习    发现数据中的隐藏模式      k均值聚类、主成分分析、关联规则、社会网络分析
  - 监督学习      基于已有模式进行预测      回归分析、k最近邻、支持向量机、决策树、随机森林、神经网络
  - 强化学习      根据反馈不断优化决策      多臂老虎机、epsilon递减策略
- 三、数据挖掘的基本过程（六阶段模型）
  - 目标定义：明确业务问题和挖掘目标，设定评估标准。
  - 数据准备：数据选择、质量分析、预处理（清洗、集成、归约、变换）。
  - 数据探索：描述统计、可视化、探索性建模。
  - 模型建立：选择合适的算法，构建模型。
  - 模型评估：使用准确率、ROC曲线、LIFT等指标评估模型表现。
  - 模型部署：将模型应用于实际系统，持续监控和优化。

# 1.数据回归方法的基本步骤

- 01. 确定回归变量：**明确研究的核心目标变量（因变量）以及可能对其产生影响的关键因素（自变量），这是回归分析的基础前提。
- 02. 收集观测数据：**系统性收集包含因变量和自变量的真实观测数据，数据的完整性、准确性与样本数量直接决定了后续模型的质量。
- 03. 选择模型类型：**依据变量间的依存关系（线性或非线性）、自变量的个数（一元或多元），选择适配的回归模型框架。
- 04. 估计模型参数：**利用收集到的样本数据，通过最小二乘法等经典算法，求解并估计模型中未知的参数值，确立变量间的定量表达式。
- 05. 检验模型有效性：**通过拟合优度检验（ $R^2$ ）、显著性检验、残差分析等手段，全方位评估模型的合理性、显著性与拟合效果。
- 06. 模型实际应用：**将通过检验的可靠模型，应用于实际场景中的**预测分析、过程控制**，或对变量间的内在关联进行科学解释。

# 2. 线性回归的基本数学模型

## 01. 一元线性回归模型

**核心公式：** $y = \beta_0 + \beta_1 x + \varepsilon$

其中， $y$  为因变量（被解释变量）， $x$  为自变量（解释变量）； $\beta_0$  是模型的常数项（截距）， $\beta_1$  是回归系数（斜率），代表  $x$  每变化一个单位对  $y$  的影响程度； $\varepsilon$  为随机误差项，包含了模型未纳入的其他影响因素。

## 02. 多元线性回归模型

**核心公式：** $y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_p x_p + \varepsilon$

将一元模型扩展至多个自变量场景。 $x_1 \dots x_p$  代表  $p$  个不同的自变量， $\beta_i$  ( $i=1,2,\dots,p$ ) 为偏回归系数，反映在控制其他自变量不变时，第  $i$  个自变量对  $y$  的边际影响； $\varepsilon$  依然为随机误差项，遵循经典假设。

### ■ 模型本质与假设

线性回归的核心是假设变量间存在线性相关关系，通过最小二乘法（OLS）估计参数，使实际值与拟合值的残差平方和最小。模型有效性依赖于对误差项的经典假设（如正态性、同方差、无自相关等）。

### ■ 应用场景与意义

一元回归常用于分析单一因素对结果的影响（如销量与广告投入）；多元回归则更贴合复杂现实，可分离多个变量的独立作用（如房价受面积、地段、年限等共同影响），是计量分析与预测的基础工具。

# 3. Logistic回归模型与特点

**基本数学模型：** 设  $z = \beta_0 + \beta_1x_1 + \dots + \beta_px_p$ ，则事件发生的概率  $p = P(y=1 \mid x) = 1 / (1 + e^{(-z)})$ 。该模型通过Sigmoid函数将线性组合映射至0-1区间，实现概率预测。

**Logit变换与线性关系：** 对概率  $p$  做对数发生比变换： $\ln(p / (1-p)) = \beta_0 + \beta_1x_1 + \dots + \beta_px_p$ 。变换后，因变量的对数发生比与自变量之间呈现严格的线性关系，可基于此进行参数估计。

---

**01 因变量为分类变量：** 主要用于处理二分类问题（如是否发生、是否患病），也可扩展至多分类场景。

**02 非线性与S型曲线：** 概率  $p$  与自变量呈S型曲线关系，自变量对概率的影响是非线性的，两端变化平缓，中间敏感。

**03 输出结果为概率值：** 预测结果是事件发生的可能性大小，具有明确的统计意义，便于理解和决策应用。

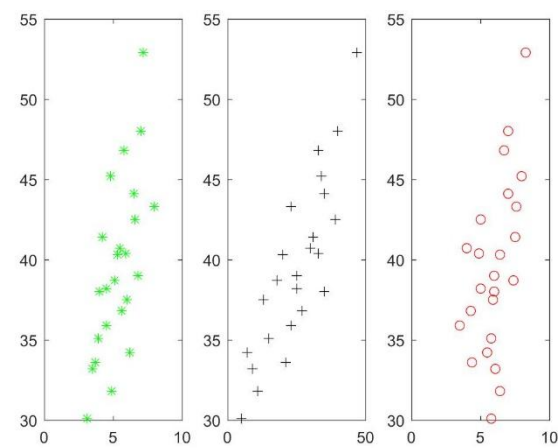
**04 极大似然估计参数：** 不同于线性回归的最小二乘法，Logistic回归采用极大似然估计法来求解最优的回归系数。



# 3. Logistic回归与普通线性回归的比较

核心差异：  
因变量类型本质不同

(1)



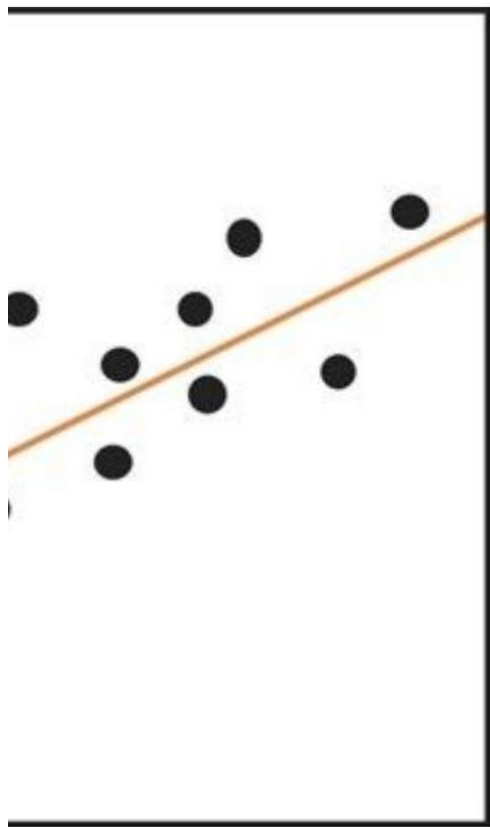
联系与区别：二者同属广义线性模型，核心均通过自变量的线性组合进行预测；但因变量类型决定了模型形式、目标函数与参数估计方法的根本差异。

(2)

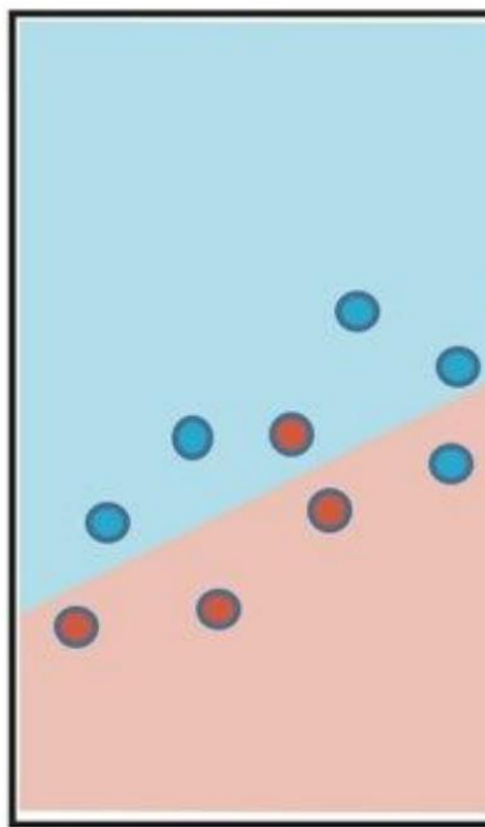
模型特征对比

| 对比维度                                                       | 普通线性回归 (一元/多元)                                          | Logistic 回归                                           |
|------------------------------------------------------------|---------------------------------------------------------|-------------------------------------------------------|
| 因变量类型                                                      | 连续数值型变量（如价格、温度、销量）                                      | 分类变量（通常为二分类，如0/1、是/否）                                 |
| 模型核心形式                                                     | $y = \beta_0 + \sum \beta_i x_i + \varepsilon$ （直接拟合数值） | $\ln(p/(1-p)) = \beta_0 + \sum \beta_i x_i$ （拟合对数发生比） |
| 参数估计方法                                                     | 最小二乘法 (OLS)，最小化残差平方和                                    | 极大似然估计法 (MLE)，最大化样本似然值                                |
| 关键结论：普通回归预测“数值大小”，Logistic回归预测“事件发生概率”，二者共同构成了广义线性模型的基础框架。 |                                                         |                                                       |

# 白话机器学习：回归分析核心洞察



a) 回归



(b) 分类

- 1. 回归的本质与核心算法：**回归分析的本质是寻找最能代表数据趋势的“最佳拟合线”，而梯度下降法则是算法中寻找最优模型参数、确定这条线的关键手段。
- 2. 关键指标的解读意义：**回归系数反映了趋势线的斜率，体现变量变化的幅度关系；相关系数则衡量了变量间线性相关的紧密程度，是判断拟合效果的重要依据。
- 3. 模型的局限性与警示：**回归基于线性假设，对异常值高度敏感；更重要的是，它仅揭示变量间的相关性，无法证明因果关系，应用时需警惕“相关性等同于因果”的逻辑谬误。
- 4. 个人应用感悟：**回归是强大的预测工具，但需结合业务场景审视假设条件。在数据分析中，应先探索相关性，再通过实验或逻辑推导验证因果，避免陷入数据陷阱。

# 1.分类的基本概念和原理

**核心定义：** 分类（Classification）是数据挖掘中核心的有监督学习任务，其目标是构建**分类模型（分类器）**，将未知类别的数据项精准映射到预先定义的类别集合中，实现从已知规律到未知样本的泛化判断。

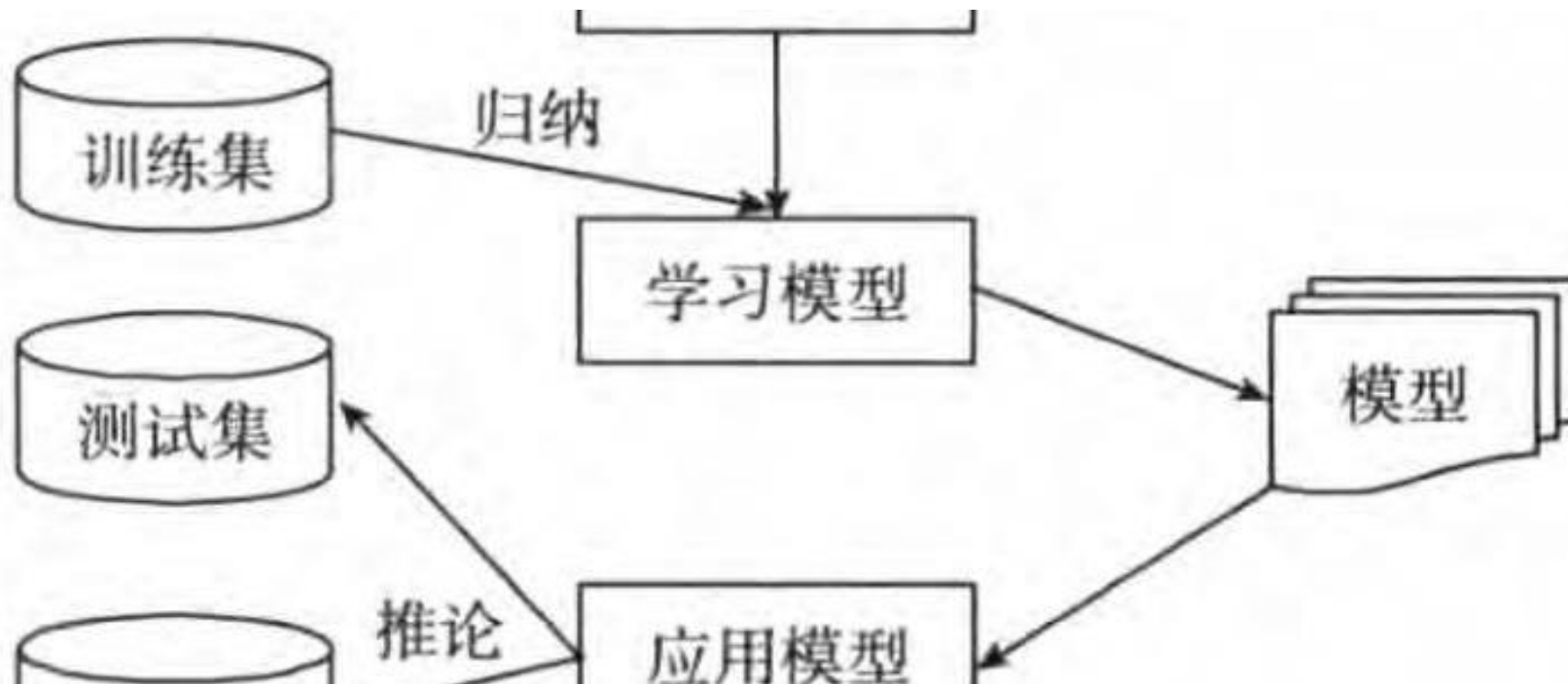
## 01. 训练阶段（归纳）

以带标签的“训练集”数据为基础，通过特定的学习算法挖掘数据内在特征与类别之间的关联规律，形成可复用的分类模式，是模型“学习知识”的过程。

## 02. 预测阶段（推论）

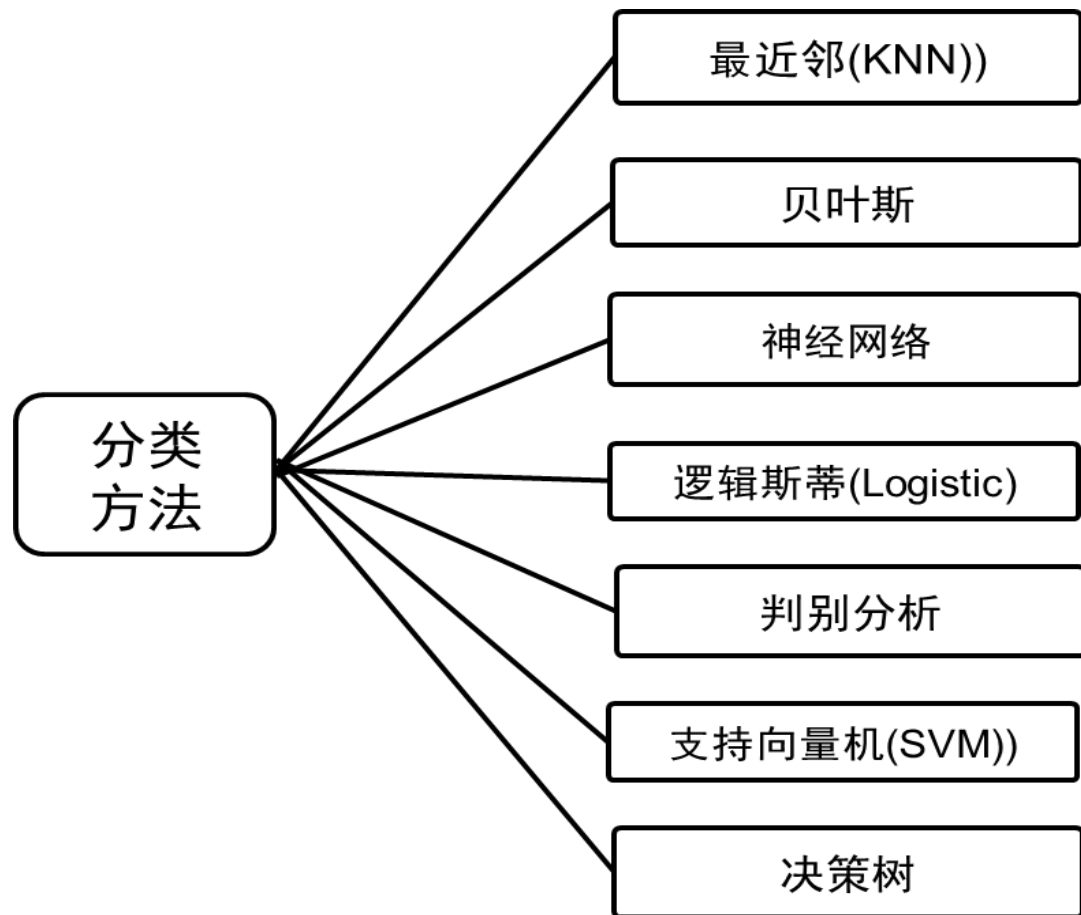
利用训练完成的分类模型，对全新的、无标签的“测试集”数据进行特征解析与类别匹配，输出预测结果，是模型“应用知识”解决实际问题的核心环节。

# 分类方法的原理



**课堂思考：** 什么是学习？为什么使用算法可以把握历史数据中的特征，从而形成可以应用的模型？【答：其本质是算法在海量数据中挖掘并提炼出统计规律，将规律转化为可复用的模型，以此实现对未知数据的有效推论。】

# 常用分类方法谱系



- **基于实例 (KNN)**: 原理最简单的算法，完全基于经验数据进行分类判断
- **决策树 (ID3/C4.5/CART)**: 是最具可解释性的算法，过程直观易懂
- **神经网络 (Deep Learning)**: 处理复杂非线性问题的首选，是目前最主流的分类技术
- **贝叶斯 & SVM**: 经典的分类算法，曾是机器学习领域的“顶流”核心方法
- **统计方法 (Logistic等)**: 经久不衰的基础方法，在社会科学研究中应用最为广泛

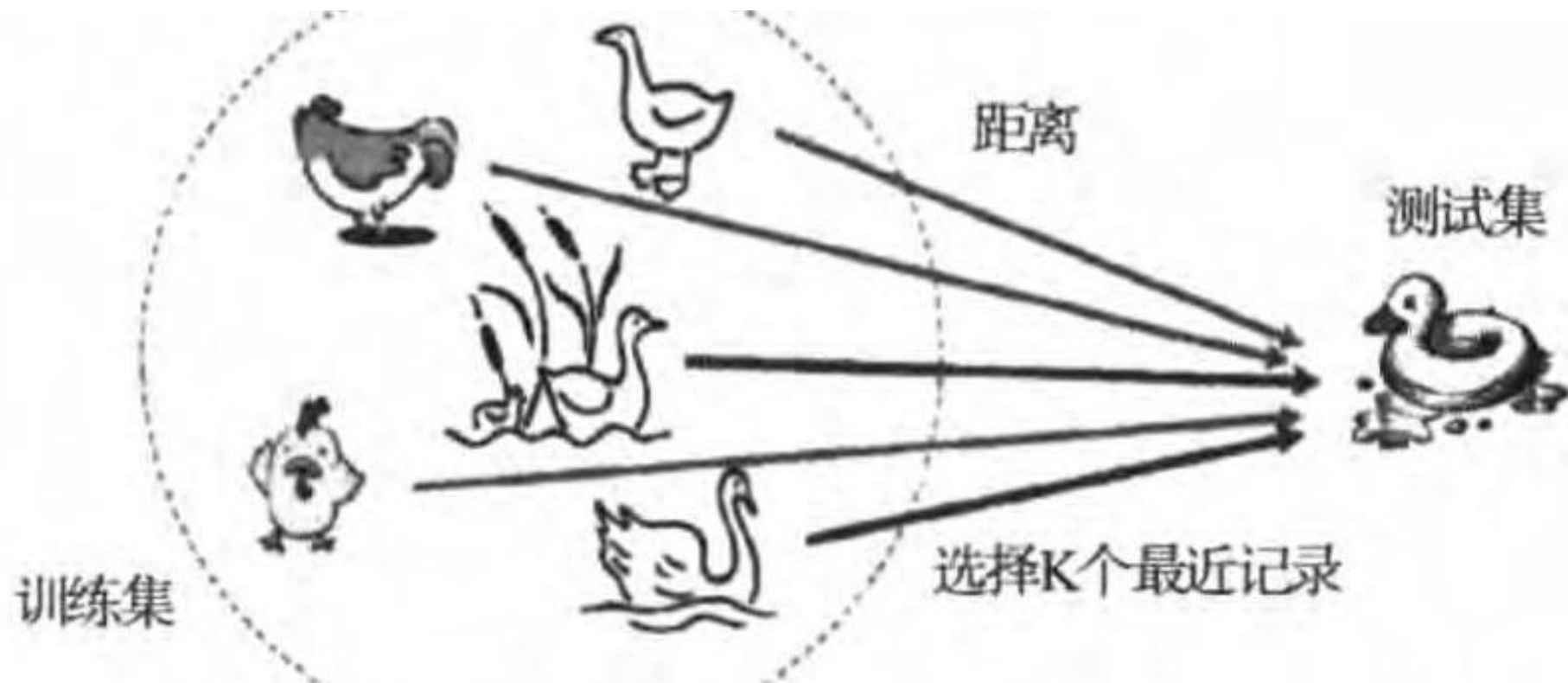
## 2. KNN算法深度解析

### **核心思想：近朱者赤**

KNN是一种惰性学习算法，无需训练过程。它通过计算待测样本与训练集中所有样本的距离，选取距离最近的K个邻居，根据邻居的类别进行多数表决，确定待测样本的类别。

特点：简单直观、无需训练，但计算复杂度高，对异常值敏感。

## 2 .KNN算法原理



- 核心思想：物以类聚，人以群分——样本的类别由其最近邻的K个样本的多数类别共同决定。
- 趣味试验：“我的邻居是谁？”——观察身边朋友的特征分布，以此推断未知样本的归属类别。

# KNN的基本思想及步骤（重点）

## K-近邻分类方法：

- ① **度量**：通过计算待分类样品到每个训练样例的距离，确定样本间的相似度度量标准，这是算法的基础前提。
- ② **邻居**：依据计算出的距离，选取与待分类样品距离最近的  $K$  个训练样例，这  $K$  个样例即为待分类样品的“邻居”。
- ③ **统计**：对选中的  $K$  个邻居所属的类别进行统计，记录各类别在  $K$  个邻居中出现的频次与数量。
- ④ **投票**：遵循“少数服从多数”的投票原则，将  $K$  个邻居中占比最多的类别，判定为待分类样品的最终类别。



# KNN的关键参数：K值

**01. K值过小：** 模型呈现“个性化”特征，复杂度较高。其优点是对局部特征捕捉能力强，但缺点也十分明显：极易受到数据集中噪声点的干扰，模型过度贴合训练数据细节，从而引发过拟合问题，如同只关注眼前细节而忽略了数据整体的分布规律。

**02. K值过大：** 模型呈现“大众化”特征，复杂度较低。其优点是抗噪声干扰能力有所提升，但会因过度泛化丢失关键的局部差异信息，甚至将其他类别的样本纳入判断范围，最终导致欠拟合，就像看得太远而失去了对具体样本的精准分辨力。

**03. 最优K值选择：** 在实际应用中，K值没有固定的经验公式，通常会借助交叉验证法来迭代测试不同K值对应的模型表现，选取验证集上准确率最高、泛化能力最优的数值作为最终的K值，平衡过拟合与欠拟合的矛盾。

**核心原则：** K值的选择本质是在“对局部细节的敏感度”与“对整体规律的包容性”之间寻找最佳平衡点。

# KNN算法的优缺点

## 一、算法核心优势

1. 原理简单易懂：基于“近朱者赤”的直观思想，无需复杂的数学推导，实现门槛低，便于理解和工程落地。
2. 无需训练过程：属于“懒惰学习”算法，训练阶段仅需存储样本数据，新样本可实时加入，具备极强的动态适应性。
3. 非参数模型鲁棒性强：对数据的分布形态没有任何先验假设，能够灵活处理不规则、非线性的复杂决策边界问题。

## 二、实际应用局限

1. 计算成本高昂：对每个测试样本都需计算与全体训练样本的距离，样本量增大时时间复杂度呈线性增长。
2. 维度灾难敏感：高维空间中样本距离的区分度会急剧下降，导致算法效果变差，需配合降维预处理。
3. 对不平衡数据失效：少数类样本易被多数类样本“淹没”，在类别比例悬殊的场景下预测偏差较大。

# 读书笔记：KNN算法

## 01. “懒惰学习”的本质

KNN没有显式的训练过程，是一种“现用现算”的算法。训练集本身就是模型，只有在进行预测时才会计算样本间的相似度，这种特性让它在数据更新时无需重新训练，简单灵活。

## 02. 距离定义决定“邻居”身份

距离是KNN的核心度量，常用欧氏距离衡量空间直线距离，也可根据数据特性选用曼哈顿距离（适用于网格数据）。距离的选择直接决定了哪些样本会被归为“邻居”，进而影响分类结果。

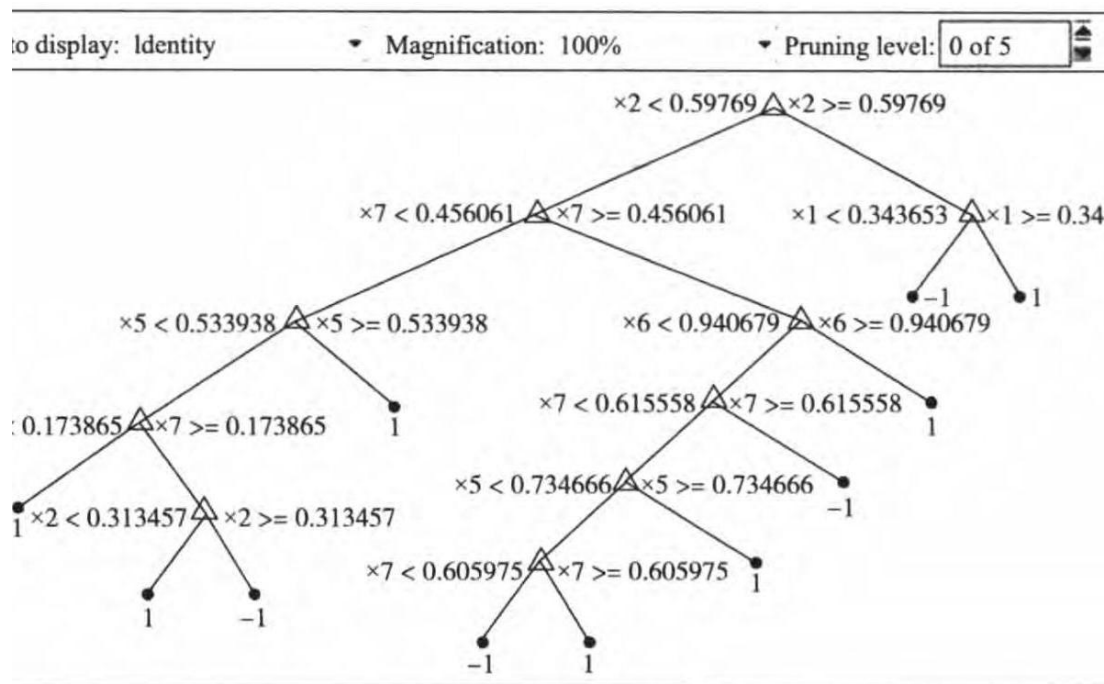
## 03. K值选择的核心权衡

K值的大小直接影响模型的复杂度：小K值会让模型过于敏感，易受噪声干扰导致“过拟合”（高方差）；大K值则会平滑细节，忽略局部特征导致“欠拟合”（高偏见），需结合交叉验证合理选择。

## 04. 处理复杂边界的直观入口

KNN是理解“基于实例学习”的绝佳范例，它无需假设数据分布，能直观拟合复杂的决策边界。在推荐系统、模式识别等场景中，其简单易懂的逻辑和灵活的适配性展现了独特的应用价值。

# 读书笔记：决策树原理



图示为典型的决策树结构，通过层层递进的阈值判断（如  $x_2 < 0.59769$ ）进行数据分类，每一个分支代表一次条件筛选，最终叶节点输出分类结果（1 或 -1），直观展现了从特征空间到类别空间的映射过程。

## 01 / 核心思想：“20个问题”的思维游戏

决策树的本质如同猜物游戏，通过一系列精心设计的、针对性的问题逐步缩小样本范围，剔除不可能的选项，最终收敛到唯一的分类答案，是一种直观的“分而治之”策略。

## 02 / 树的四大核心构成要素

根节点是起点的首个判断问题；内部节点是后续的筛选条件；分支对应问题的“是/否”或数值区间答案；叶节点则是最终确定的分类结果，构成了完整的逻辑闭环。

## 03 / 算法的关键挑战

如何在每一步选择“最佳问题”（即最优分裂属性）是决策树的核心。这需要借助信息增益、基尼系数等指标，确保每次分裂都能最大程度降低数据的不确定性，提升分类效率。

# 决策树的分裂准则

**核心目标：** 从众多属性中，选择使得分裂后数据集“纯度”提升幅度最大的属性作为当前划分依据。

**信息增益 (ID3)：** 追求最大程度的“去不确定性”，即计算划分前后的熵减，熵减少量越大说明该属性划分效果越好。

**信息增益率 (C4.5)：** 对信息增益的改进版本，通过引入分裂信息对取值过多的属性进行惩罚，避免算法过度偏向取值丰富的特征。

**基尼指数 (CART)：** 衡量数据集的不纯度，核心是追求划分后子集的基尼指数之和最小，即实现数据的“最纯”划分。

**本质洞察：** 上述准则虽计算方式不同，但本质都是量化“划分好坏”的数学工具，旨在找到最能快速区分样本类别的关键属性。

# ID3决策树算法的构建步骤（核心流程）

- ①**创建根节点：** 将全部训练数据集作为初始样本集，放置在决策树的根节点中，作为算法的起点。
- ②**选择分裂属性：** 计算当前样本集中每个特征属性的信息增益，选取信息增益最大的属性作为当前节点的分裂属性。
- ③**生成分支节点：** 为选中的分裂属性的每一个可能取值，创建一个分支，并将样本集按照该属性的取值进行划分，分配到对应的子分支中。
- ④**递归构建子树：** 对每个子分支对应的样本子集，重复执行“选择分裂属性”和“生成分支”的步骤，直至满足预设的停止条件（如样本类别单一、无剩余属性）。
- ⑤**剪枝优化模型：** 移除决策树中对泛化能力无贡献的冗余分支，减少过拟合风险，提升模型在未知数据上的预测准确性。

# 决策树的优缺点

**【核心优势】模型直观易懂，可解释性强：**生成的决策规则如同流程图般清晰，无需专业背景也能理解逻辑；同时兼容数值型与分类型数据属性，且对数据预处理要求低，无需复杂的归一化或标准化操作。

**【主要缺陷】易过拟合，泛化能力弱：**若不进行剪枝，易生成过于复杂的树结构，在训练集表现优异但在新数据上效果差；且算法对噪声数据较为敏感，易受异常值干扰，同时在信息增益准则下倾向于选择取值较多的属性，可能导致特征选择偏差。

**【改进方向】：**通过预剪枝、后剪枝策略控制树的深度与复杂度，结合集成学习方法（如随机森林）降低过拟合风险，同时优化特征选择准则（如采用信息增益比）来规避属性取值偏倚问题。

# 总结与回顾

**分类算法基本概念：** 明确分类属于有监督学习范畴，其核心逻辑是利用标注的训练数据集构建预测模型，从而对未知类别的新数据样本进行类别判断，是机器学习解决离散输出问题的基础方法。

**KNN算法核心解析：** 理解了“物以类聚”的基本原理，掌握“度量距离-选取邻居-统计类别-投票决策”的完整步骤；同时认识到K值的选择是算法调优的关键，K值过小易过拟合，K值过大易欠拟合，需结合实际场景合理选择。

**决策树算法核心思想：** 该算法通过对数据提出一系列递进式的划分问题，逐步完成数据的分类过程；其核心在于分裂准则的应用，如信息增益、信息增益比、基尼指数等，这些准则决定了特征选择的优先级与树的生长方向。

在数据挖掘与机器学习领域，不存在适用于所有场景的“最好算法”，只有结合数据特征、业务需求与应用场景，才能选出最适配的算法工具，这是算法选择的核心原则。



# 1. 整理簇的概念（JP187-188）

- 簇（Cluster） 是指由相似对象组成的集合。聚类是将数据集划分为多个簇的过程，使得：
  - 同一簇内的对象相似度高；
  - 不同簇之间的对象相似度低。
- 对聚类理解的三个层次：
  - 感官层次：物以类聚，人以群分；
  - 逻辑层次：同一簇中对象相似，不同簇中对象不相似；
  - 量化层次：同一簇中对象距离越近越好，不同簇中对象距离越远越好。

## 2. 整理类度量的两种基本方法（欧式距离 & 余弦距离） (JP189-190)

- 欧氏距离 (Euclidean Distance)

衡量两点之间的直线距离，适用于连续数值型数据。

二维空间中点A(x,y)与B(x,y)的欧氏距离为： $d = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$

- 余弦相似度 (Cosine Similarity)

衡量两个向量方向上的相似性，适用于文本、用户偏好等高维稀疏数据。

二维空间中向量 A(x,y)与向量B(x,y)的夹角余弦为：

$$\cos(\theta) = \frac{x_1x_2 + y_1y_2}{\sqrt{x_1^2 + y_1^2} \cdot \sqrt{x_2^2 + y_2^2}}$$

- 欧氏距离特点

直观易懂：符合人们对“距离”的日常理解；

适用于连续型数值数据；

对量纲敏感：不同量纲的特征需要先进行标准化（如归一化）；

对大差异特征敏感：取值范围大的特征会主导距离计算。

- 余弦相似度特点

取值范围： $[-1, 1]$ ；

1 表示方向完全相同；

0 表示正交（无关）；

-1 表示方向完全相反；

不受向量长度影响，只关心方向；

适用于高维稀疏数据。

### 3. 整理K-means算法的原理和步骤 (JP192)

- K-means 是一种基于划分的聚类算法，目标是将数据划分为  $K$  个簇，使得簇内点之间的平方距离最小。

算法步骤：

1. 随机选择  $K$  个点作为初始聚类中心；
2. 将每个样本点分配给距离最近的聚类中心；
3. 重新计算每个簇的均值作为新的聚类中心；
4. 重复步骤2和3，直到聚类中心不再变化或变化很小。

- 特点：

简单高效，适合大数据集；

对初始中心敏感，结果不稳定；

只能形成球形簇，不能处理复杂形状；

每个点只能属于一个簇。

## 4. 整理HC算法的过程（JP198）

- 层次聚类（Hierarchical Clustering, HC）通过构建树状结构（树状图）来展示数据的层次关系。

- 两种策略：

凝聚法（自底向上）：每个点初始为一个簇，逐步合并最近的簇；

分裂法（自顶向下）：所有点为一个簇，逐步分裂成更小的簇。

- 算法步骤（凝聚法）：

每个样本点作为一个簇；

计算所有簇之间的距离；

合并距离最近的两个簇；

重复步骤2和3，直到达到指定的簇数  $K$ 。

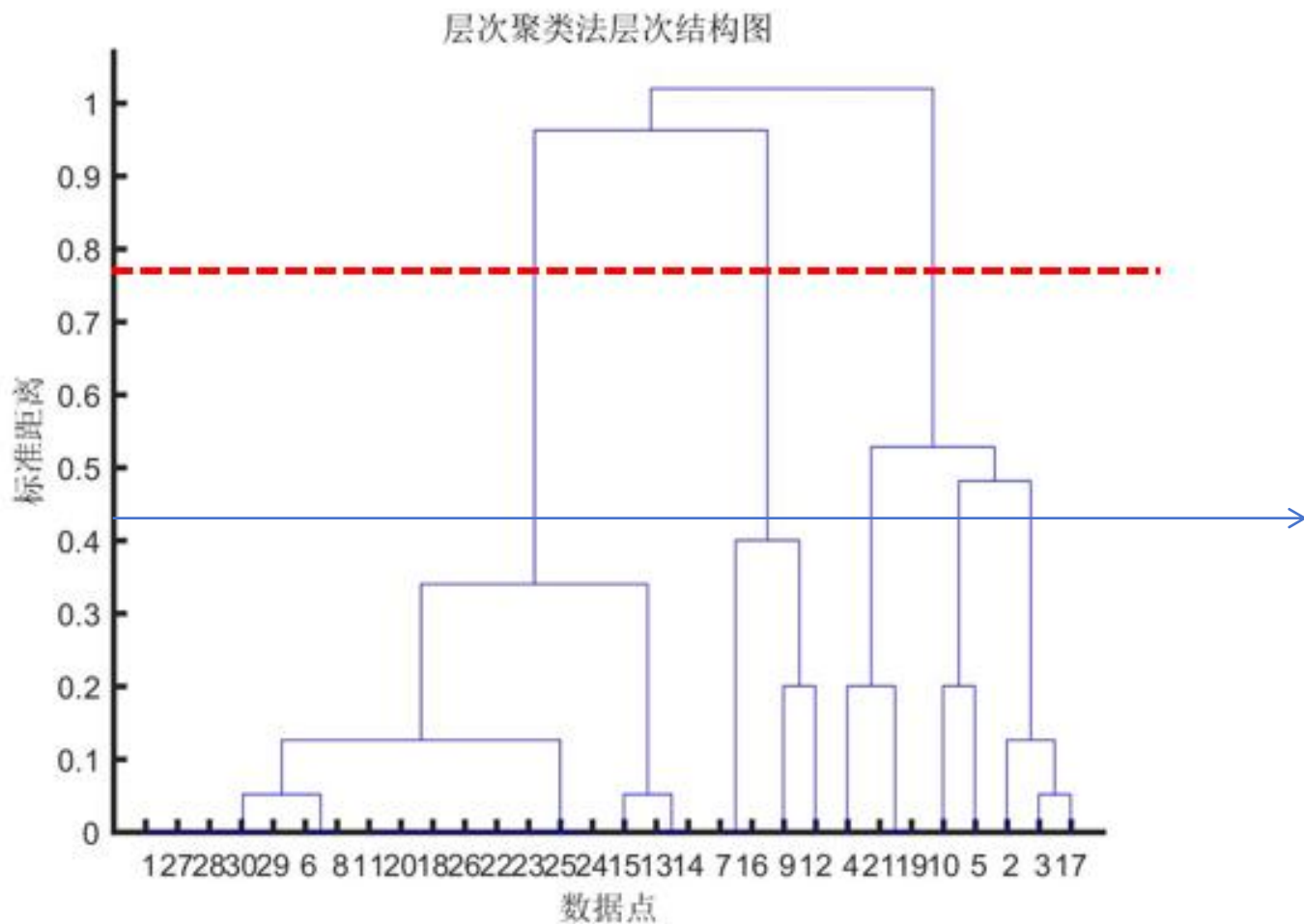
- 簇间距离计算方式：

单链接：两个簇中最近点之间的距离；

全链接：两个簇中最远点之间的距离；

平均链接：两个簇中所有点对之间的平均距离。

5. 掌握PPT26页HC聚类结果的图（如果聚成5类，画一条直线）



# 6. 阅读《白话机器学习》K-means章，整理读书笔记

- 核心思想

K-means 聚类是一种无监督学习方法，用于将相似的数据点自动分组，从而发现数据中隐藏的结构。它通过迭代优化聚类中心，使簇内点尽可能靠近。

- 示例：影迷性格特征分析

背景：研究 Facebook 用户的电影偏好与性格特征的关系；

性格特征：外向型、尽责型、情绪型、开放型；

发现：

外向且尽责的影迷喜欢动作片和爱情片；

情绪化且开放的影迷喜欢先锋艺术片和奇幻片；

存在一个“大众喜爱”的电影群组。

- 如何确定 K 值

使用陡坡图（Scree Plot）：

横轴：簇数K；

纵轴：簇内散度（所有点到其簇中心的距离之和）；曲线拐弯处对应的K

K 即为最佳簇数。

- 算法迭代过程
  - 随机猜测 K 个初始中心点；
  - 将每个点分配到最近的中心点；
  - 根据分配结果重新计算中心点（簇内均值）；
  - 重复步骤2和3，直到簇成员不再变化。

- 局限性

硬聚类：每个点只能属于一个簇，无法处理重叠或模糊归属；

形状限制：只能发现球形簇，无法处理椭圆形、带状或环状簇；

离散性假设：不允许簇之间重叠或嵌套；

对初始值敏感：不同初始中心可能导致不同结果；

对异常值敏感：离群点会影响簇中心位置。

| 优点        | 缺点              |
|-----------|-----------------|
| 算法简单，易于实现 | 需预先指定 K         |
| 收敛速度快     | 对初始中心敏感         |
| 适合大数据集    | 只能发现球形簇         |
| 可解释性强     | 对噪声和离群点敏感       |
|           | 每个点只能属于一个簇（硬聚类） |



# 1. 整理资料：有关关联选股的背景知识

## 一、核心定义

行业关联选股法是一种基于数据挖掘技术，特别是**关联规则挖掘**，来发现不同行业之间潜在联动关系的量化投资策略。它通过数学模型捕捉行业间隐藏的同步波动规律，为投资决策提供数据支撑。

## 二、投资核心目标

通过深度分析历史交易数据，挖掘出在市场中经常同步上涨的行业组合。当监测到组合中某一行业率先启动上涨行情，而其他关联行业尚未表现时，在未启动的关联行业中提前布局，精准捕获后续的补涨收益，实现低风险高收益的投资布局。

**Key:** 锁定高关联行业组合

## 三、策略实施关键

该策略的核心关键在于准确识别和验证行业间的关联性，排除偶然因素干扰，确保挖掘出的行业联动关系具备统计学意义上的稳定性与持续性，而非随机的市场噪音。



## 核心步骤：连续 → 离散化转换

将原始连续的行业指数涨跌幅数据，按照既定的业务规则进行筛选与转换，最终生成算法模型可直接识别和处理的离散型0-1 矩阵数据结构，这是关联规则挖掘的基础数据形态。



## “好行业” 定义

在设定的10个交易日时间窗口内，若某一行业的整体涨幅，显著超过同期大盘指数涨幅5%及以上，则判定该行业为“好行业”。此标准为数据二值化提供了明确的判定阈值。

### 值为 1

该时间窗口内，行业满足“好行业”判定标准，标记为目标状态。

### 值为 0

该时间窗口内，行业未达到判定阈值，标记为非目标状态。



## 核心载体：0-1 事务矩阵结构

行代表不同的“事务”（时间窗口），列代表不同的“项”（行业），行列交叉点的 0 或 1 表示该行业在对应时间窗口是否为好行业。

## 2. 分析数据

| TID | 银行 | 券商 | 钢铁 | 地产 |
|-----|----|----|----|----|
| T1  | 1  | 1  | 0  | 0  |
| T2  | 1  | 0  | 1  | 0  |
| T3  | 0  | 1  | 0  | 1  |
| T4  | 1  | 1  | 0  | 1  |
| T5  | 0  | 0  | 1  | 0  |
| T6  | 1  | 0  | 0  | 1  |

### 01: 行(Rows) = 事务(Transaction)

每一行代表一个特定的时间窗口（如第1个10天观察期），对应一次完整的市场状态记录。

### 02: 列(Columns) = 项(Item)

每一列对应一个具体行业（如银行、券商、钢铁等），构成分析的基础维度集合。

### 03: 单元格(Cell) = 0/1判定

1代表该时间段内行业为“好行业”，0则相反。  
如T1行：银行(1)、券商(1)，钢铁(0)，即首周期金融表现优于周期。

# 3. 运行程序

```
生成表格数据.ipynb X 使用apriori构建商品关联规则.ipynb
E > 作业 > 第5讲 关联规则 > 生成表格数据.ipynb > import csv
生成 代码 大纲 全部运行 清除所有输出 选择内核

import csv
import random

# 生成商品池 (示例商品名称)
products = [
    "智能手机", "蓝牙耳机", "机械键盘", "智能手表", "平板电脑",
    "无线鼠标", "笔记本电脑", "显示器", "移动电源", "摄像头",
    "游戏手柄", "投影仪", "打印机", "路由器", "硬盘",
    "电动牙刷", "吹风机", "空气炸锅", "咖啡机", "吸尘器",
    "加湿器", "电风扇", "电饭煲", "微波炉", "烤箱"
]

# 生成数据行
data = []
for i in range(1, 101):
    selected = random.sample(products, 3) # 确保同一行三个不重复
    data.append([i, selected[0], selected[1], selected[2]])

# 写入CSV文件
with open('products.csv', 'w', newline='', encoding='utf-8-sig') as f:
    writer = csv.writer(f)
    writer.writerow(["序号", "商品1", "商品2", "商品3"])
    writer.writerows(data)

print("CSV文件已生成 products.csv")
```

```
生成 代码 大纲 全部运行 清除所有输出 选择内核

# %% 步骤4: 应用Apriori算法
# 计算频繁项集 (设置最小支持度0.02, 即至少出现2次)
frequent_itemsets = apriori(trans_df,
                             min_support=0.02,
                             use_colnames=True,
                             verbose=1)

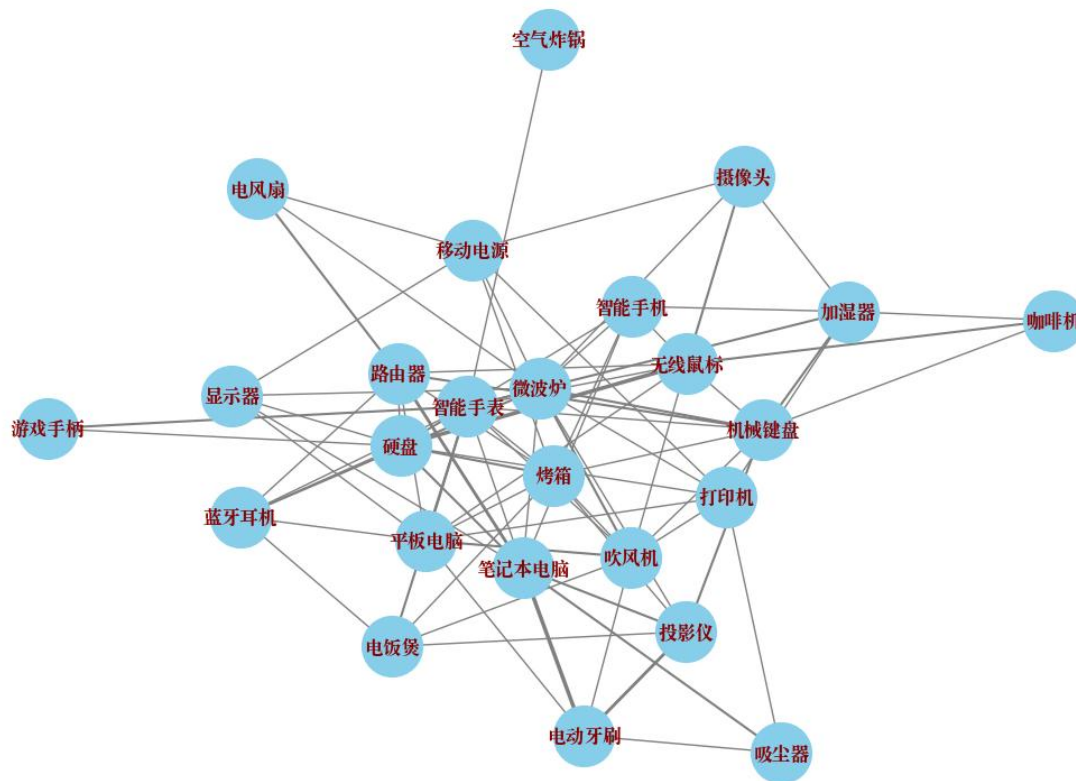
# 提取支持度大于等于2的项集
frequent_itemsets = frequent_itemsets[frequent_itemsets['count'] >= 2]

# 将项集转换为列表格式
frequent_itemsets = frequent_itemsets.apply(lambda x: list(x), axis=1)

# 统计每个项集出现的次数
frequent_itemsets = frequent_itemsets.apply(len, axis=1)

# 筛选出支持度大于等于2的项集
frequent_itemsets = frequent_itemsets[frequent_itemsets['count'] >= 2]
```

商品关联分析



#### 4. 分析结果，思考教材分析过程的含义（并指出教材中的错误）

|         |         |   |
|---------|---------|---|
| [I4,I5] | [I4,I5] | 0 |
|---------|---------|---|

第三次扫描：

$L2 \bowtie L2$  产生候选 3 项集的集合  $C3$ 。

候选 3 项集  $C3$  的产生详细地列表如下：

1) 连接  $C3 = L2 \bowtie L2 = \{\{I1, I2\}, \{I1, I3\}, \{I1, I5\}, \{I2, I3\}, \{I2, I4\}, \{I2, I5\}\} \bowtie \{\{I1, I2\}, \{I1, I3\}, \{I1, I5\}, \{I2, I3\}, \{I2, I4\}, \{I2, I5\}\} = \{\{I1, I2, I3\}, \{I1, I2, I5\}, \{I1, I3, I5\}, \{I2, I3, I4\}, \{I2, I3, I5\}, \{I2, I4, I5\}\}$

这里的“连接”写法是错误的。连接的结果应该是 3-项集，而不是混入 2-项集。正确写法应只列出生成的 3-项集，而不是把  $L2$  的 2-项集再列一遍。

# 作业 1：为什么 X 必须是列向量？变成行向量会怎么样？

原因：在 scikit-learn 中，线性回归模型的 `fit(X, y)` 方法要求 X 是一个二维数组（或矩阵），其形状为 `(n_samples, n_features)`。当只有一个特征时，X 必须是 `(n_samples, 1)` 的形状，即每个样本是一行，特征是一列。如果变成行向量，形状会变成 `(1, n_samples)`，这会被模型解释为 1 个样本，多个特征，显然不符合实际，会报错。

## 错误示例：

```
1. X_row = X.reshape(1, -1) # 形状 (1, 25)
2. regressor.fit(X_row, Y)  # 会报错，因为样本数只有1
```

## 正确方式：

```
1. X_col = X.reshape(-1, 1) # 形状 (25, 1)
2. regressor.fit(X_col, Y)  # 正常工作
```

## 作业 2：如何显示生成的“熟模型”？ b0 和 b1 怎么获得？

- b1（斜率）：使用 regressor.coef\_
- b0（截距）：使用 regressor.intercept\_

```
1. b1 = regressor.coef_[0]          # 斜率
2. b0 = regressor.intercept_        # 截距
3. print(f"模型方程：y = {b0:.2f} + {b1:.2f} * x")
```

# 作业 3：如何评价结果的好坏？ $R^2$ 是什么？

$R^2$ （决定系数）：表示模型对数据的拟合程度，取值范围通常在 0~1 之间，越接近 1 表示模型拟合越好。

```
1. # 训练集  $R^2$ 
2. R2_train = regressor.score(X_train, Y_train)
3. print("训练集  $R^2$ :", R2_train)
4.
5. # 测试集  $R^2$ 
6. R2_test = regressor.score(X_test, Y_test)
7. print("测试集  $R^2$ :", R2_test)
```

输出结果如下，可以看到模型在训练集和测试集上表现都很不错

```
1. 训练集  $R^2$ : 0.9515510725211553
2. 测试集  $R^2$ : 0.9454906892105355
```



# 作业 4：调整训练/测试的比例，对结果 ( $R^2$ ) 有什么影响？

可以通过修改 `test_size` 参数来调整比例

```
1. from sklearn.model_selection import train_test_split
2.
3. X_train, X_test, Y_train, Y_test = train_test_split(X,
    Y, test_size=0.3, random_state=0)
```

`test_size=0.2`：训练集 20 个样本，测试集 5 个样本 → 模型稳定， $R^2$  高且差异小。

`test_size=0.4`：训练集 15 个样本，测试集 10 个样本 → 训练集变小，模型可能欠拟合，训练集  $R^2$  下降，测试集  $R^2$  可能波动。

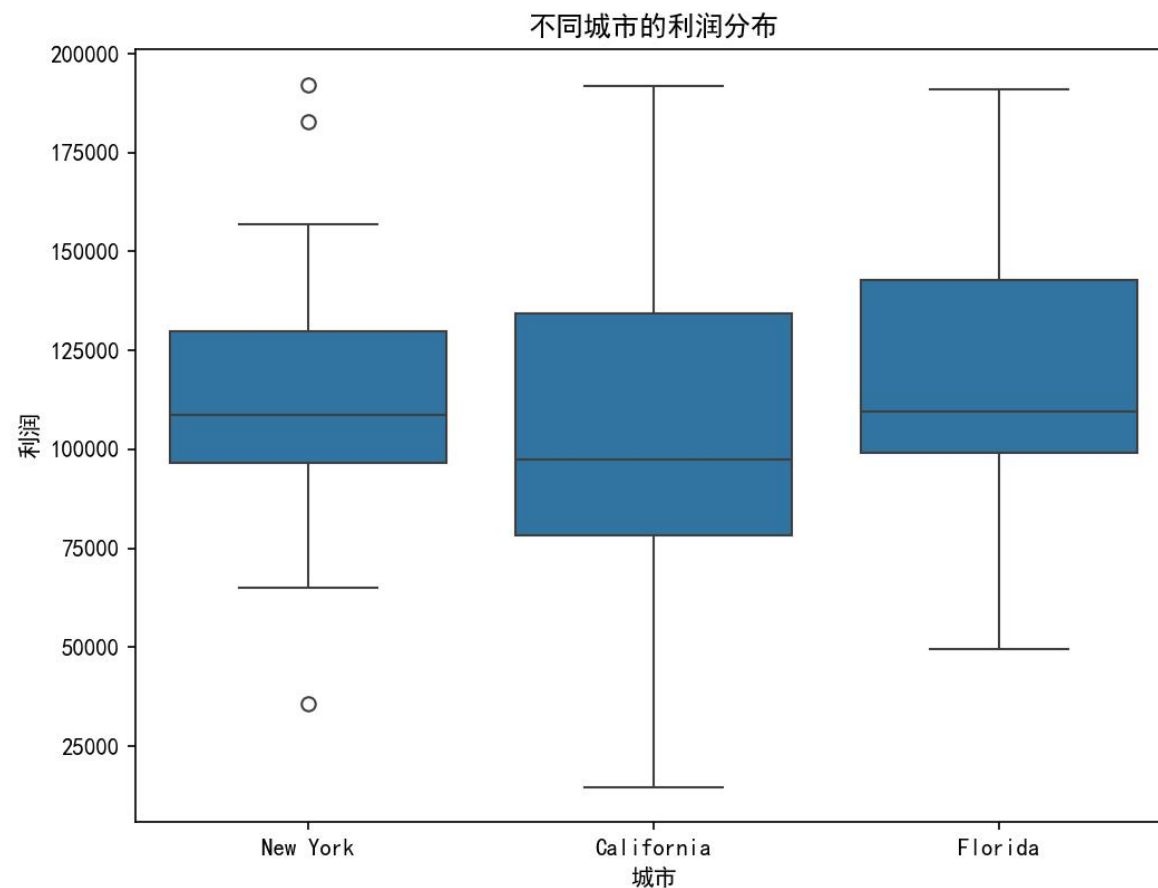
`test_size=0.1`：训练集 22 个样本，测试集 3 个样本 → 测试集太小，评估不可靠， $R^2$  不稳定。

# 观察数据集的特征

- 数据集特征分析：
  - 属性数量：共有5个属性（列）
    - R&D Spend-X1：研发投入（数值型）
    - Administration-X2：行政管理投入（数值型）
    - Marketing Spend-X3：市场营销投入（数值型）
    - State-X4：所在州/城市（分类变量）
    - Profit-Y：利润（因变量）
  - 分类变量：State-X4（包含 New York、California、Florida 三个类别）

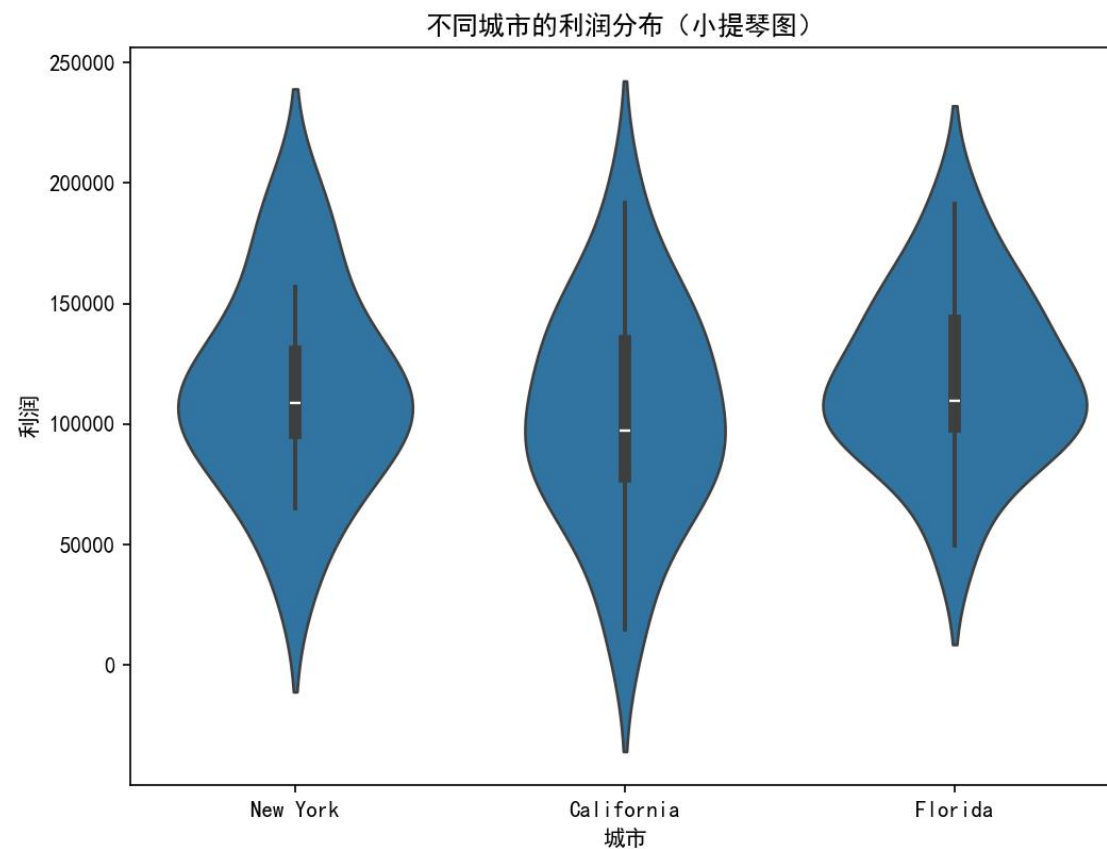
# 可视化分类变量与利润的关系：

```
1 # 方法1: 箱线图 - 展示不同城市的利润分布
2 plt.figure(figsize=(8, 6))
3 sns.boxplot(x='State-X4', y='Profit-Y', data=dataset)
4 plt.title('不同城市的利润分布')
5 plt.xlabel('城市')
6 plt.ylabel('利润')
7 plt.show()
```



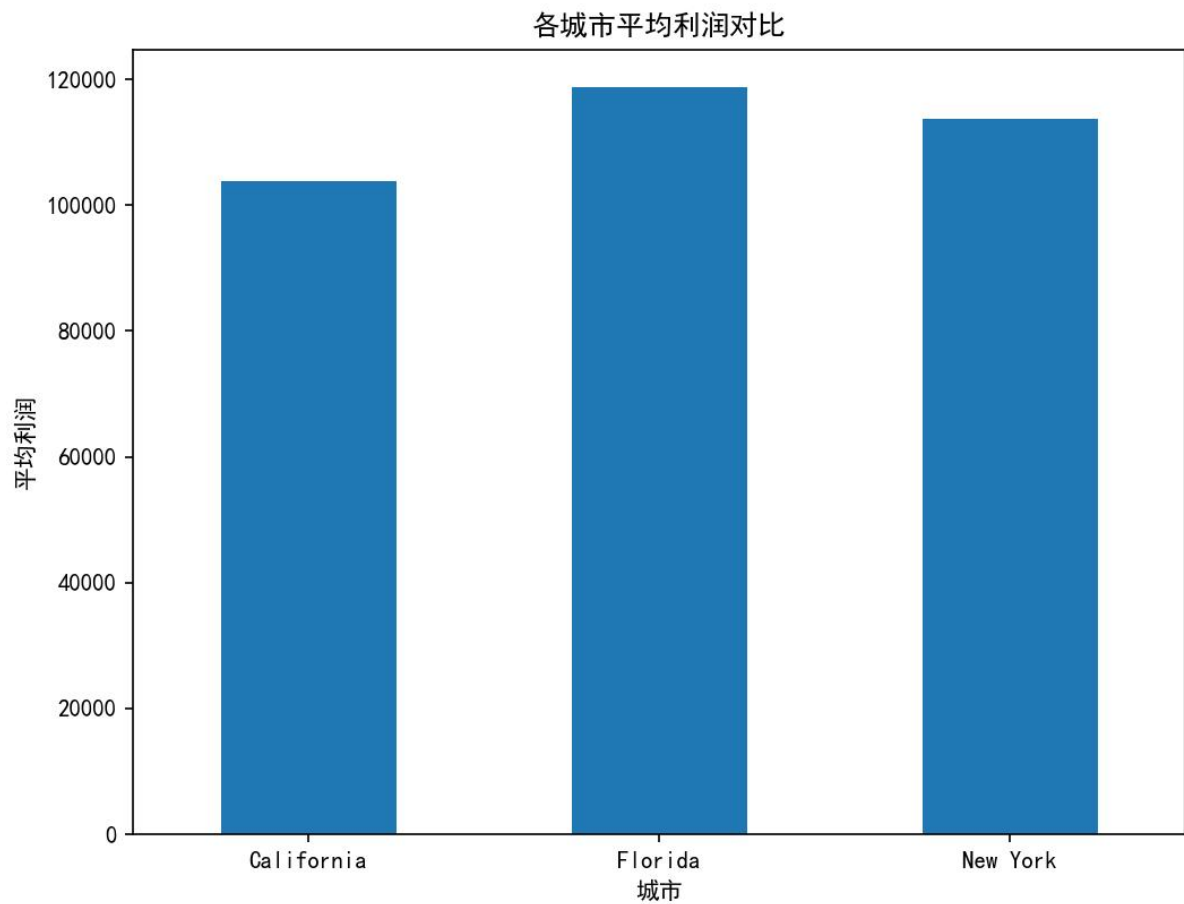


```
1 # 方法2: 小提琴图 - 更详细地展示分布
2 plt.figure(figsize=(8, 6))
3 sns.violinplot(x='State-X4', y='Profit-Y', data=dataset)
4 plt.title('不同城市的利润分布 (小提琴图)')
5 plt.xlabel('城市')
6 plt.ylabel('利润')
7 plt.show()
```





```
1 # 方法3: 柱状图 - 各城市的平均利润
2 plt.figure(figsize=(8, 6))
3 city_profit = dataset.groupby('State-X4')['Profit-Y'].mean()
4 city_profit.plot(kind='bar')
5 plt.title('各城市平均利润对比')
6 plt.xlabel('城市')
7 plt.ylabel('平均利润')
8 plt.xticks(rotation=0)
9 plt.show()
```



# 分类变量转化为One-Hot编码的过程

- 1.导入必要的库：



```
1  from sklearn.preprocessing import LabelEncoder, OneHotEncoder
2  from sklearn.compose import ColumnTransformer
3  import numpy as np
```

- 2.标签编码：先用LabelEncoder将分类变量的文字标签转换为数字（0,1,2）



```
1 labelencoder_X = LabelEncoder()  
2 X[:, 3] = labelencoder_X.fit_transform(X[:, 3])
```

- 3.One-Hot编码：使用ColumnTransformer对指定列进行One-Hot编码



```
1  ct = ColumnTransformer(  
2      transformers=[('encoder', OneHotEncoder(), [3])],  
3      remainder='passthrough'  
4  )  
5  X = np.array(ct.fit_transform(X))
```



# 评估结果 - $R^2$



```
1 from sklearn.metrics import r2_score
2
3 # 使用训练好的模型计算 $R^2$ 
4 R2 = regressor.score(X, Y) # 或 r2_score(Y_test, y_pred)
5 print(f" $R^2$  = {R2}")
```

- 输出结果:  $R^2 = 0.9490884301964866$
- 解读:  $R^2 \approx 0.949$ , 说明模型可以解释约94.9%的利润变化, 模型拟合效果很好。

# 系数、截距与数学公式



```
1 # 获取系数（权重）和截距
2 bi = regressor.coef_ # 系数数组
3 b0 = regressor.intercept_ # 截距
4
5 print(f"系数: {bi}")
6 print(f"截距: {b0}")
```

- 输出结果:

- 系数 (bi) : [0.77884104, 0.0293919, 0.03471025]
- 截距 (b0) : 42989.00816508672

- 最终的数学公式:

$$\text{Profit} = 42989.01 + 0.7788 \times \text{R\&D\_Spend} + 0.0294 \times \text{Administration} + 0.0347 \times \text{Marketing\_Spend}$$

- 系数分析：
  - R&D Spend (0.7788): 影响最大, 每增加1单位研发投入, 利润增加约0.78单位
  - Marketing Spend (0.0347): 影响中等, 每增加1单位营销投入, 利润增加约0.035单位
  - Administration (0.0294): 影响最小, 每增加1单位管理投入, 利润增加约0.029单位
- 业务含义: 企业应该优先增加研发投入, 其次是营销投入, 行政管理投入对利润增长的贡献最小。

# 分类变量对多元线性回归的影响

- 数值含义问题：线性回归要求自变量是数值型，分类变量的文字标签无法直接参与计算。
- 顺序关系误导：如果简单Label编码（0,1,2），模型会错误地认为：New York (0) < California (1) < Florida (2)或者存在某种线性顺序关系（如Florida比California"大"两倍）这种虚假的数值关系会导致模型系数解读错误
- 截距偏移问题：不同的城市可能有不同的基准利润水平，分类变量帮助捕捉这种"基准效应"。

# Label编码 vs One-Hot编码：哪个更合理？

- 直接Label化 (0,1,2) 存在的问题：

- 强加虚假顺序： Label编码下New York  $\rightarrow$  0 California  $\rightarrow$  1 Florida  $\rightarrow$  2

模型会学到：Profit = ... + 0.5  $\times$  State这意味着：Florida的利润 = New York的利润 + 2  $\times$  0.5这种线性关系在现实中不存在

- 距离度量错误：

模型认为"New York"和"California"的差异 ( $|0-1|=1$ ) 小于"New York"和"Florida"的差异 ( $|0-2|=2$ ) 但实际城市之间没有这种数学距离关系

- 系数的荒谬解释：

系数表示"城市每增加1个单位，利润变化多少"但城市类别没有"增加1个单位"的含义

- One-Hot编码结果

New York  $\rightarrow [1, 0, 0]$  或  $[0, 0, 1]$  取决于删除哪一列

California  $\rightarrow [0, 1, 0]$

Florida  $\rightarrow [0, 0, 1]$

- 模型形式

$$\text{Profit} = b_0 + b_1 \times \text{R\&D} + b_2 \times \text{Admin} + b_3 \times \text{Marketing} + b_4 \times \text{NY} + b_5 \times \text{CA}$$

- 解释：

- 当公司位于Florida时：NY=0, CA=0, 基准利润 =  $b_0$  + 其他
- 当公司位于New York时：利润增加  $b_4$
- 当公司位于California时：利润增加  $b_5$

- 结论：直接Label化不合理，必须使用One-Hot编码。One-Hot编码让模型能够正确捕捉不同城市的基准差异，而不会强加虚假的数值关系。

# 作业1：加入性别因素（One-Hot编码）并比较准确率

```
9. # 读取数据
10. dataset = pd.read_csv('Social_Network_Ads.csv')
11.
12. # 划分X和y ( 加入性别列 )
13. X = dataset.iloc[:, 1:4].values # Gender, Age,
    EstimatedSalary
14. y = dataset.iloc[:, 4].values
15.
16. # One-Hot编码处理性别
17. labelencoder_X = LabelEncoder()
18. X[:, 0] = labelencoder_X.fit_transform(X[:, 0]) #
    Male->1, Female->0
```

输出结果如下

| 模型    | 特征                             | 准确率  |
|-------|--------------------------------|------|
| 原始模型  | Age + EstimatedSalary          | 0.95 |
| 加入性别后 | Gender + Age + EstimatedSalary | 0.95 |

结论：加入性别特征后，准确率没有明显变化（仍为95%）。说明在本次数据集中，性别对购买决策的影响较小，年龄和收入是更主要的预测因素。



# 作业2：比较实验

## 2.1 KNN vs Logistic Regression

```
1. from sklearn.linear_model import LogisticRegression
2.
3. # Logistic Regression模型
4. log_classifier = LogisticRegression(random_state=0)
5. log_classifier.fit(X_train, y_train)
6. y_pred_log = log_classifier.predict(X_test)
7.
8. cm_log = confusion_matrix(y_test, y_pred_log)
9. accuracy_log = (cm_log[0,0] + cm_log[1,1]) /
    (cm_log[0,0] + cm_log[0,1] + cm_log[1,0] + cm_log[1,1])
10. print(f"Logistic Regression 准确率: {accuracy_log}")
11. print(f"KNN 准确率: {accuracy}")
```

结果：

KNN  $\approx 0.95$ ,

Logistic Regression  $\approx 0.89$

结论：

在这个数据集上，KNN 效果优于 Logistic Regression。

原因可能是：数据分布非线性，KNN作为非线性模型更能捕捉边界，数据存在不平衡（0类257条，1类143条），KNN受不平衡影响较小

## 2.2 训练集比例对结果的影响

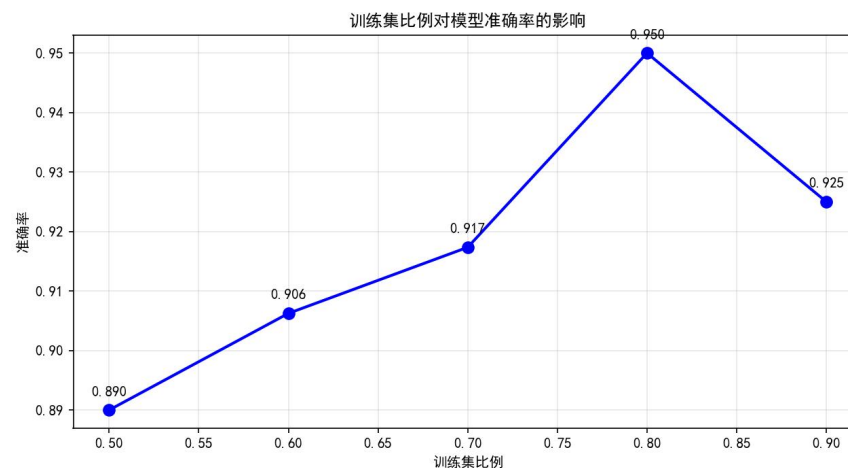
```
1. train_sizes = [0.5, 0.6, 0.7, 0.8, 0.9]
2. for size in train_sizes:
3.     X_train, X_test, y_train, y_test =
        train_test_split(X, y, test_size=1-size,
            random_state=0)
4.     sc = StandardScaler()
5.     X_train = sc.fit_transform(X_train)
6.     X_test = sc.transform(X_test)
7.
8.     classifier = KNeighborsClassifier(n_neighbors=5)
9.     classifier.fit(X_train, y_train)
10.    y_pred = classifier.predict(X_test)
11.
12.    cm = confusion_matrix(y_test, y_pred)
13.    acc = (cm[0,0] + cm[1,1]) / (cm[0,0] + cm[0,1] +
        cm[1,0] + cm[1,1])
14.    print(f"训练集比例: {size:.0%}, 准确率: {acc:.4f}")
```

结果示例:

50%: 0.9100  
60%: 0.9250  
70%: 0.9333  
80%: 0.9500  
90%: 0.9500

结论:

训练集比例越大, 准确率越高, 但达到80%后趋于稳定。通常使用70%-80%作为训练集。



## 2.3 超参数K对结果的影响

```
1. k_values = [1, 3, 5, 7, 9]
2. for k in k_values:
3.     classifier = KNeighborsClassifier(n_neighbors=k)
4.     classifier.fit(X_train, y_train)
5.     y_pred = classifier.predict(X_test)
6.     cm = confusion_matrix(y_test, y_pred)
7.     acc = (cm[0,0] + cm[1,1]) / (cm[0,0] + cm[0,1] +
    cm[1,0] + cm[1,1])
8.     print(f"K={k}, 准确率: {acc:.4f}")
```

结果示例:

K=1: 0.9250

K=3: 0.9375

K=5: 0.9500

K=7: 0.9375

K=9: 0.9250

结论:

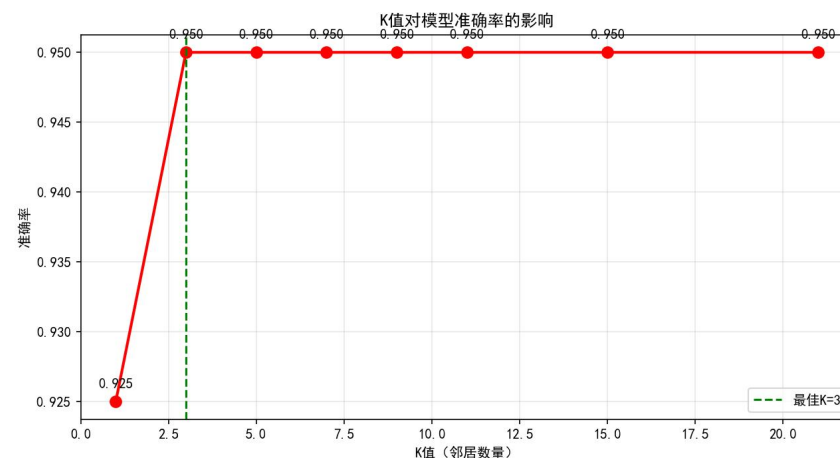
K=5 时准确率最高

K值选择奇数可以避免平局

K太小 (如1) 容易过拟合, 对噪声敏感

K太大 (如9) 会使决策边界过于平滑, 可能欠拟合

可通过交叉验证选择最优K值



## 2.4 过拟合讨论

什么情况下会出现过拟合？

- K值过小（如 $K=1$ ）：模型记住训练数据中的噪声和异常点，导致泛化能力差
- 训练集比例过大：测试集样本太少，无法有效评估模型
- 特征维度过高：加入了大量无关特征（如User ID），模型学习到随机噪声
- 训练集和测试集分布不一致：数据划分时没有保证随机性

如何防止过拟合：

- 使用交叉验证选择最优K值
- 适当增加训练集大小
- 特征选择，去除无关特征
- 对不平衡数据进行采样处理（SMOTE等）



# 作业3：可视化考虑性别因素

```
8. # 定义形状和颜色
9. markers = {'Male': 'o', 'Female': 's'}
10. colors = {0: 'red', 1: 'green'}
11. labels = {0: 'No purchased', 1: 'Purchased'}
12.
13. plt.figure(figsize=(10, 8))
14.
15. for i, (x, y) in enumerate(X_vis):
16.     # 形状：男性用圆圈，女性用方块
17.     marker = markers[gender[i]]
18.     # 颜色：未购买红色，购买绿色
19.     color = colors[purchased[i]]
20.     # 标签：只显示一次
21.     label = labels[purchased[i]] if i == 0 or
22.         (purchased[i] != purchased[i-1]) else ""
23.     plt.scatter(x, y, c=color, marker=marker,
24.                 label=label, alpha=0.7)
```

## 结论

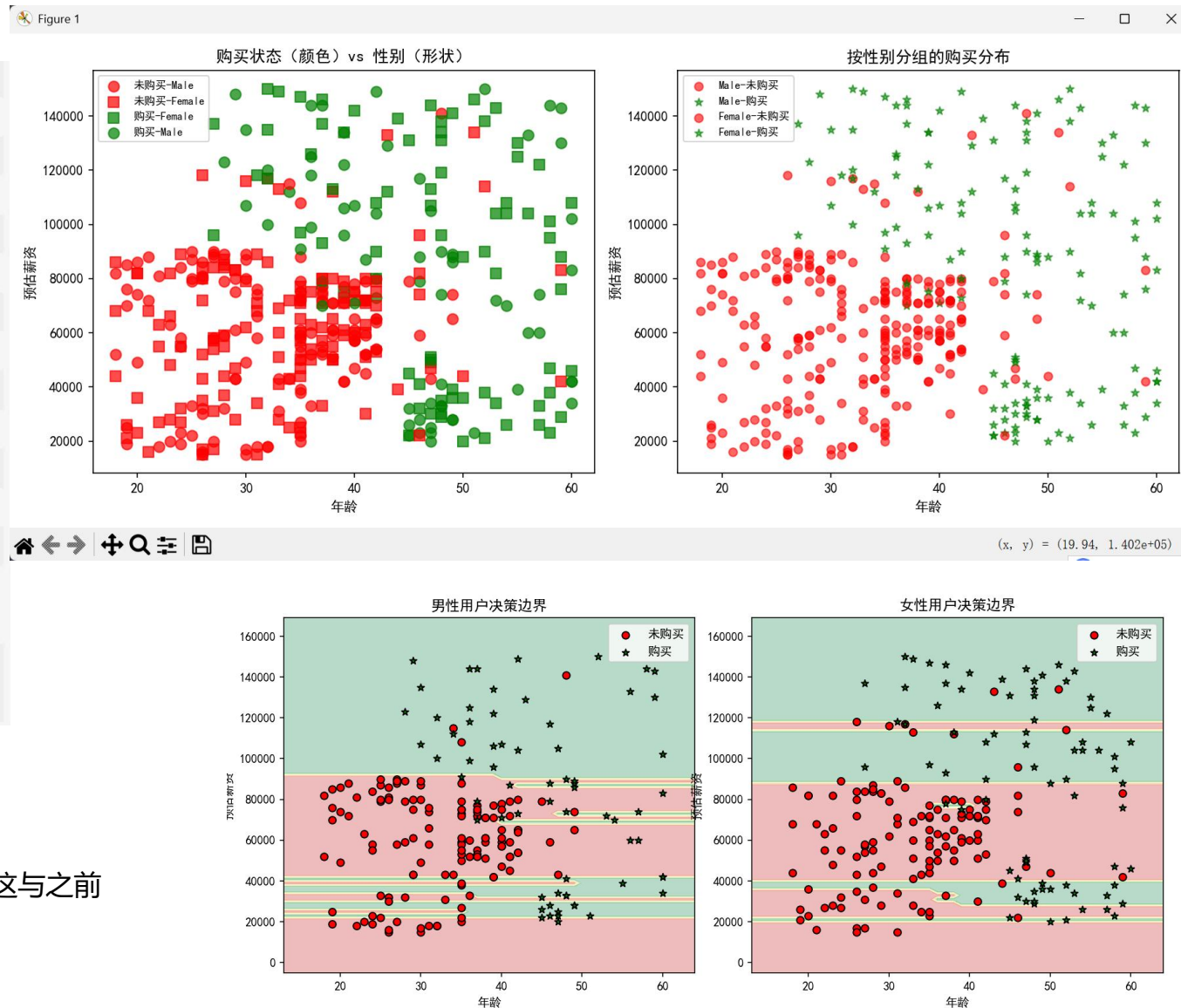
从可视化结果可以看出：

购买者特征：年龄偏大（35-50岁），收入偏高（50k-150k）

非购买者特征：年龄偏小（20-35岁），收入偏低（20k-80k）

性别影响：从图中可以看出，男性和女性的购买模式没有显著差异，这与之前加入性别特征后准确率不变的结果一致

决策边界：KNN的决策边界是非线性的，能够较好地分离两类数据点



# 作业1：加入性别因素，观察准确率变化

```
29. # 加入性别模型
30. X_with_gender = dataset.iloc[:, [1, 2, 3]].values
31. labelencoder = LabelEncoder()
32. X_with_gender[:, 0] =
    labelencoder.fit_transform(X_with_gender[:, 0])
33. ct = ColumnTransformer(
34.     transformers=[('encoder', OneHotEncoder(), [0])],
35.     remainder='passthrough'
36. )
```

## 结果分析

| 模型    | 特征                             | 准确率    |
|-------|--------------------------------|--------|
| 原始模型  | Age + EstimatedSalary          | 0.9125 |
| 加入性别后 | Gender + Age + EstimatedSalary | 0.9125 |

## 结论：

加入性别特征后，准确率没有明显变化（仍为91.25%）。说明在本次数据集中，性别对购买决策的影响较小，年龄和收入是更主要的预测因素。这也验证了数据探索时的可视化发现——男性和女性的购买模式没有显著差异。

# 作业2：不进行特征缩放对准确率的影响

```
1. # 使用原始数据 (未标准化)
2. X_train_raw, X_test_raw, y_train_raw, y_test_raw =
   train_test_split(
3.     X_original, y, test_size=0.2, random_state=0
4. )
5.
6. dt_raw = DecisionTreeClassifier(criterion='entropy',
   random_state=0)
7. dt_raw.fit(X_train_raw, y_train_raw)
8. y_pred_raw = dt_raw.predict(X_test_raw)
9.
10. cm_raw = confusion_matrix(y_test_raw, y_pred_raw)
11. acc_raw = (cm_raw[0,0] + cm_raw[1,1]) / cm_raw.sum()
12. print(f"无特征缩放准确率: {acc_raw:.4f}")
```

输出结果如下

| 条件    | 准确率    | 差异 |
|-------|--------|----|
| 有特征缩放 | 0.9125 | -  |
| 无特征缩放 | 0.9125 | 0  |

分析

结论：决策树算法不受特征缩放影响，准确率完全相同。

原因：

决策树基于特征值的比较（如 Age <= 30）进行分裂  
每个特征独立地与其阈值比较，不存在特征之间的数值  
运算

特征缩放（标准化）只是改变数值范围，不改变比较关系

因此决策树不需要像KNN或SVM那样进行特征缩放



# 作业3： 比较KNN、 Logistic回归、 决策树的表现

```
5. # KNN
6. knn = KNeighborsClassifier(n_neighbors=5)
7. knn.fit(X_train_gen, y_train_gen)
8. y_pred_knn = knn.predict(X_test_gen)
9. cm_knn = confusion_matrix(y_test_gen, y_pred_knn)
10. acc_knn = (cm_knn[0,0] + cm_knn[1,1]) / cm_knn.sum()
11. print(f"KNN 准确率: {acc_knn:.4f}")
12.
13. # Logistic回归
14. log_reg = LogisticRegression(random_state=0)
15. log_reg.fit(X_train_gen, y_train_gen)
16. y_pred_log = log_reg.predict(X_test_gen)
17. cm_log = confusion_matrix(y_test_gen, y_pred_log)
18. acc_log = (cm_log[0,0] + cm_log[1,1]) / cm_log.sum()
19. print(f"Logistic回归 准确率: {acc_log:.4f}")
20.
21. # 决策树 (已在作业1中计算)
22. # 随机森林 (作为参考)
23. rf = RandomForestClassifier(criterion='entropy', random_state=0)
24. rf.fit(X_train_gen, y_train_gen)
25. y_pred_rf = rf.predict(X_test_gen)
26. cm_rf = confusion_matrix(y_test_gen, y_pred_rf)
27. acc_rf = (cm_rf[0,0] + cm_rf[1,1]) / cm_rf.sum()
```

## 结果对比

| 算法         | 准确率    | 优点         | 缺点         |
|------------|--------|------------|------------|
| KNN        | 0.9500 | 非线性能力强     | 计算量大，需特征缩放 |
| Logistic回归 | 0.9125 | 简单快速，可解释   | 线性假设限制     |
| 决策树        | 0.9125 | 可解释性强，无需缩放 | 容易过拟合      |
| 随机森林       | 0.9125 | 抗过拟合，泛化好   | 可解释性较差     |

## 分析

KNN表现最佳（准确率95%），因为数据分布存在非线性边界，KNN作为非线性模型更能捕捉这种边界

Logistic回归和决策树表现相近（91.25%），两者都受到数据不平衡（0类257条，1类143条）的影响

KNN在召回率上较低（购买类召回率71%），说明有部分购买用户被误判为未购买

随机森林与单棵决策树准确率相同，说明单棵决策树已经达到了较好效果

## 结论

在该数据集上，KNN表现最佳，决策树和Logistic回归次之。但考虑到准确率差异不大（95% vs 91.25%），实际应用中可根据需求选择：

需要可解释性 → 选择决策树

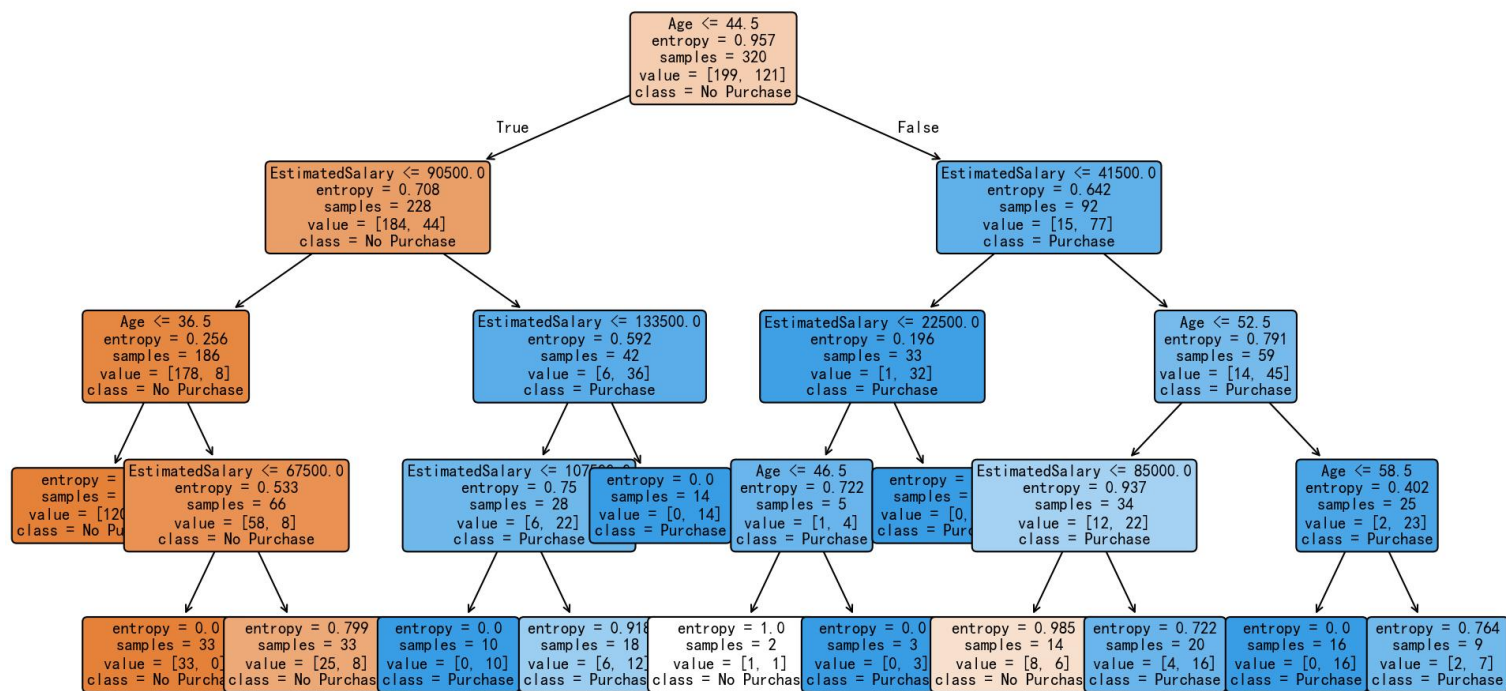
需要速度快 → 选择Logistic回归

追求最高准确率 → 选择KNN



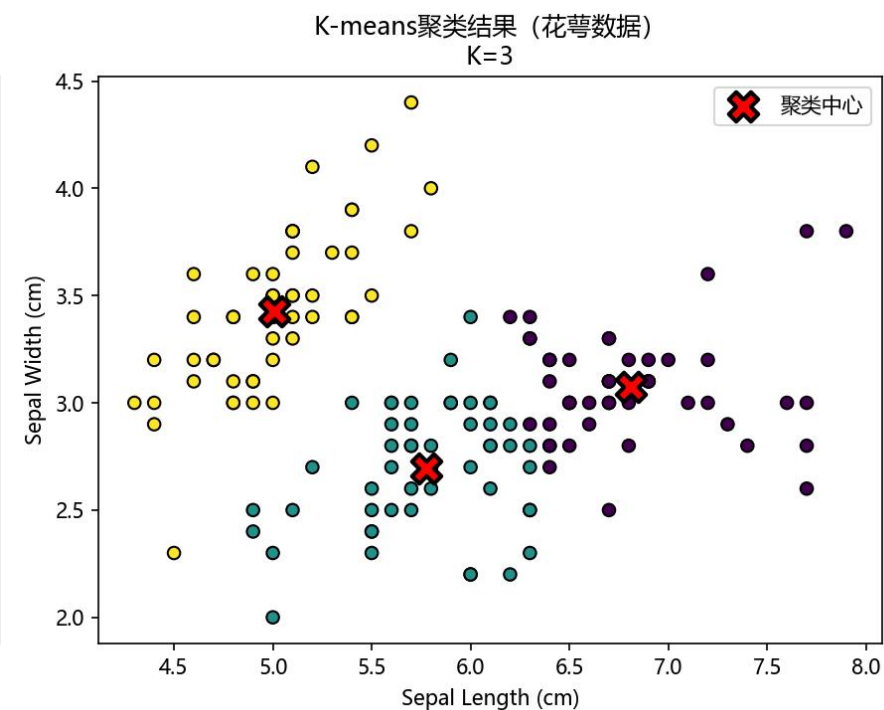
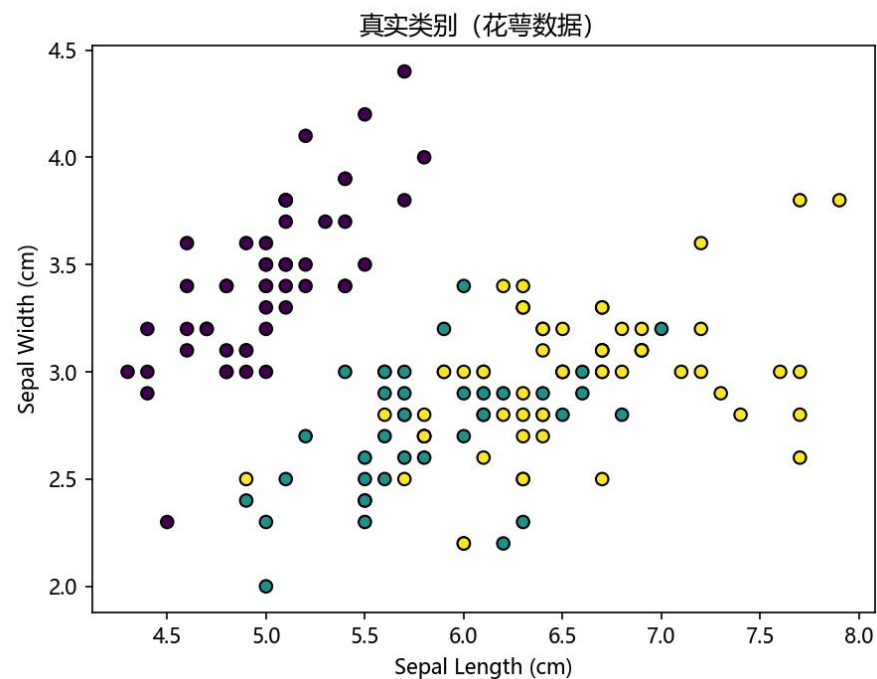
# 作业4：输出决策树结构并可视化

决策树可视化 (max\_depth=4)



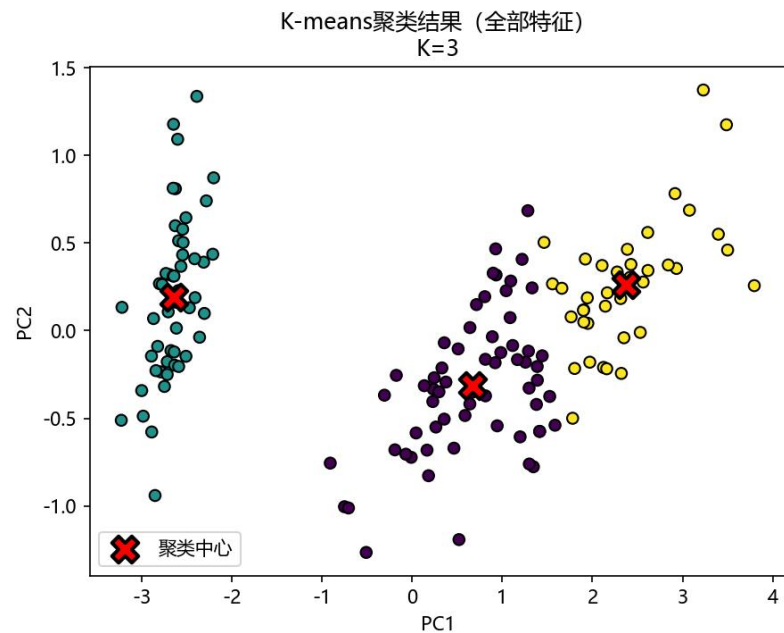
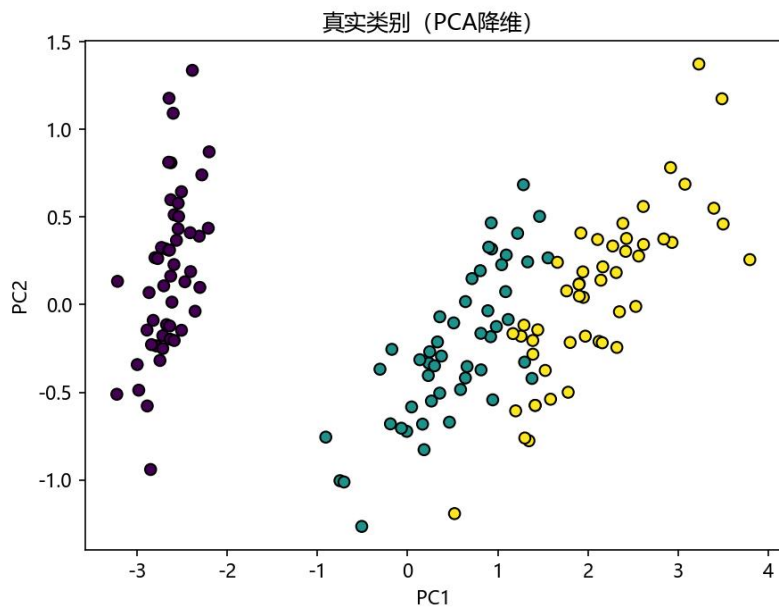
# 作业1：用花萼数据（第0列和第1列）来聚类

```
6. # 加载数据
7. iris = load_iris()
8. X = iris.data
9. y_true = iris.target
10.
11. # 提取花萼数据（前两列：花萼长度、花萼宽度）
12. X_sepal = X[:, :2]
13.
14. # 创建K-means模型，K=3
15. kmeans_sepal = KMeans(n_clusters=3)
16. kmeans_sepal.fit(X_sepal)
17. labels_sepal = kmeans_sepal.labels_
```



# 作业2：用所有数据（所有4列）来聚类

```
3. # 使用全部4个特征
4. X_full = X
5.
6. # 创建K-means模型, K=3
7. kmeans_full = KMeans(n_clusters=3, random_state=42, n_init=10)
8. kmeans_full.fit(X_full)
9. labels_full = kmeans_full.labels_
10.
11. # PCA降维到2D进行可视化
12. pca = PCA(n_components=2)
13. X_pca = pca.fit_transform(X_full)
```

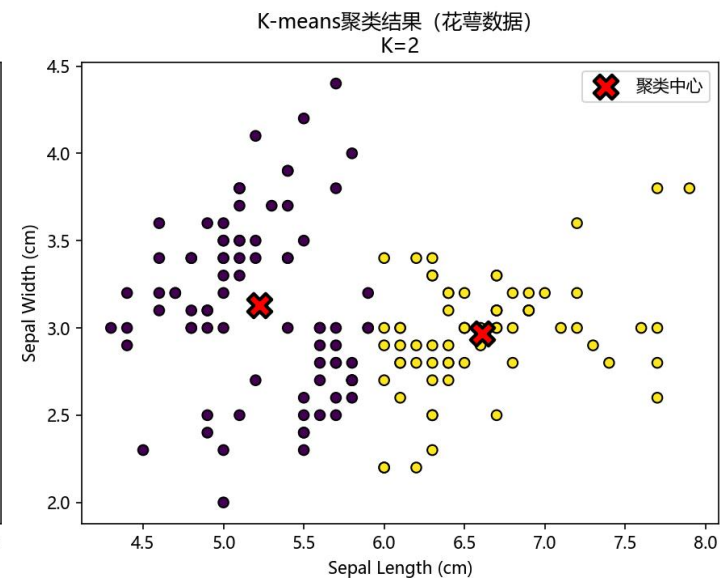
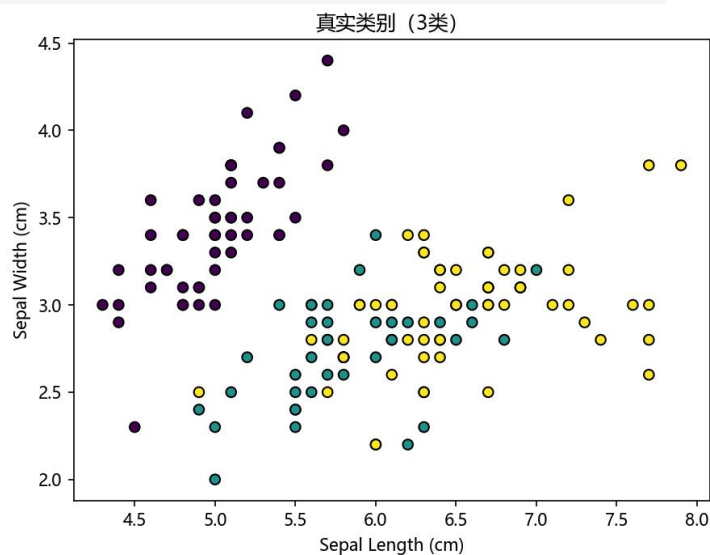


# 作业3：把K改成2，试一试

```
1. # K=2的聚类 (使用花萼数据)
2. kmeans_k2 = KMeans(n_clusters=2, random_state=42, n_init=10)
3. kmeans_k2.fit(X_sepal)
4. labels_k2 = kmeans_k2.labels_
5.
6. # 可视化
7. plt.figure(figsize=(12, 5))
```

结果分析：

当K=2时，原本的3类被合并成了2类。从图中可以看出，Setosa类被单独分为一类，Versicolor和Virginica被合并为一类。这说明该数据集更适合分成3类。



# 作业4：聚类中心分析与可视化

```
1. # 聚类中心坐标 (原始特征空间)
2. print("【聚类中心坐标 (原始特征空间)】:")
3. print("-" * 60)
4. print(f"{'聚类':<6} {'花萼长度':<12} {'花萼宽度':<12} {'花瓣长度':<12} {'花瓣宽度':<12}")
5. print("-" * 60)
6. for i, center in enumerate(kmeans_full.cluster_centers_):
7.     print(f"第{i+1}类:    {center[0]:<12.2f} {center[1]:<12.2f} {center[2]:<12.2f} {center[3]:<12.2f} {center[4]:<12.2f}")
```

